

알고리즘

Chap 4. 탐욕 알고리즘 (Greedy Approach)

■ 탐욕 알고리즘 (Greedy Approach)

■ 탐욕 알고리즘

- 선택 시점에 가장 좋아 보이는 답을 선택함으로써 최종 답에 도달한다
- 어떤 선택이든지 선택할 당시에는(locally) 최적이지만, 전체적으로(globally) 최적인 해를 얻는다는 보장은 없다
- 선택된 결정은 되돌려지지 않는다.
 - 일단 어떤 동전을 선택하기로 하면 그 동전은 영원히 거스름 돈에 포함되고, 일단 어떤 동전을 포기하기로 하면 그 동전은 영원히 거스름돈에서 제외된다
- 탐욕 알고리즘은 최적의 해를 보장하지 않는다
 - 탐욕 알고리즘으로 설계했다면, 최적의 해를 주는지 항상 점검해야 한다

■ 탐욕 알고리즘 (Greedy Approach)

- (example) 거스름돈의 동전 개수를 최소로 만드는 문제

```
while (동전이 남아있고 문제 미해결) {  
    가장 가치가 높은 동전을 선택                // 선택과정(selection procedure)  
    if (동전을 추가하여 거스름 돈의 액수를 초과) // 적절성검사(feasibility check)  
        동전을 되돌려 놓는다  
    else  
        동전을 포함시킨다  
    if (동전을 추가하여 거스름 돈의 액수와 동일) // 해답점검(solution check)  
        문제 해결  
}
```

- (example) 거스름돈의 동전 개수를 최소로 만드는 문제

- 거스름 돈 액수: 36원
- 동전의 종류: 25, 10, 10, 5, 1, 1
- 1. 25원 선택 (총액) $25 < 36$
- 2. 10원 선택 (총액) $35 < 36$
- 3. 10원 선택, 복원 (총액) $45 > 36$
- 4. 5원 선택, 복원 (총액) $40 > 36$
- 5. 1원 선택 (총액) $36 == 36$

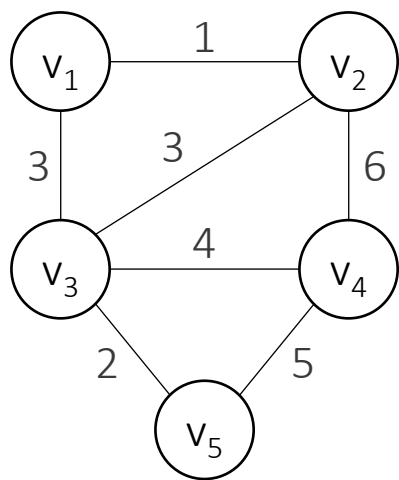
■ 탐욕 알고리즘 (Greedy Approach)

- 탐욕 알고리즘이 항상 최적의 해를 찾는다는 보장은 없다
 - (example) 12원 동전이 있는 경우, 최적해를 찾는다는 보장이 사라진다
 - 거스름 돈 액수: 16원
 - 동전의 종류: 12, 10, 5, 1, 1, 1, 1
 - 1. 12원 선택 (총액) $12 < 16$
 - 2. 10원 선택, 복원 (총액) $22 > 16$
 - 3. 5원 선택, 복원 (총액) $17 > 16$
 - 4. 1원 선택 (4번) (총액) $16 == 16$
 - 탐욕알고리즘의 결과는 5개 (12, 1, 1, 1, 1)
 - 최적의 해답은 3개 (10, 5, 1) : 탐욕 알고리즘은 최적해를 찾지 못한다.
- 탐욕 알고리즘의 절차
 - 공집합에서부터 문제의 해답이 될 때까지 원소를 반복적으로 추가
 - 선택과정(selection procedure): 현재 시점에서 최적을 선택하는 탐욕 기준에 따라서 집합에 추가할 다음 원소를 선택
 - 적절성검사(feasibility check): 업데이트된 집합이 해답이 되기에 적절한지 검사
 - 해답점검(solution check): 업데이트된 집합이 문제의 해답인지 결정

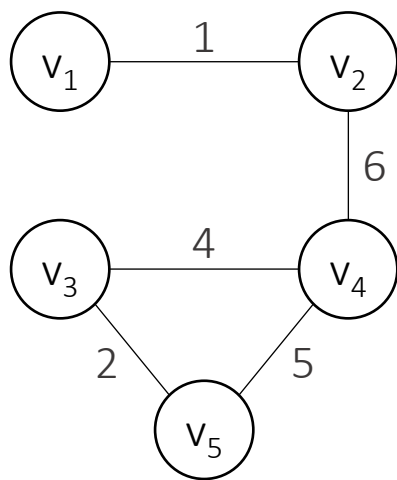
■ 최소비용 신장트리 (Minimum Spanning Trees)

■ 그래프 $G = (V, E)$

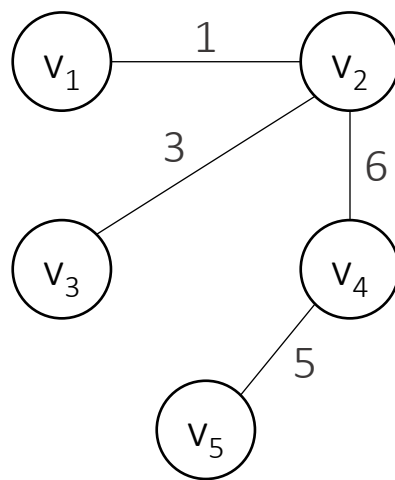
- (a)는 연결된 가중치 포함(weighted), 비방향(undirected) 그래프 G
- 비방향(undirected): edge에 방향이 없음
- Path: edge로 연결된 vertex의 배열
- 연결되어있다(connected): 비방향 그래프에서 모든 vertex의 쌍에 경로가 있음
- 단순순환경로(simple cycle): vertex에서 시작해서 다시 자신으로 돌아오는 경로
- 비순환적(acyclic): 단순순환경로가 없는 비방향 그래프 (ex. (c), (d))



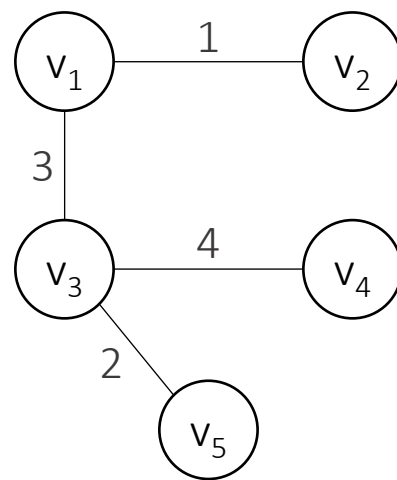
(a) connected, weighted, undirected graph G



(b) If (v_4, v_5) were removed from this subgraph, the graph would remain connected



(c) A spanning tree for G



(d) A minimum spanning tree for G

■ 최소비용 신장트리 (Minimum Spanning Trees)

■ Tree

- Free tree: 비순환적이며 연결되어 있는 비방향 그래프
- Rooted tree: vertex 하나를 root로 지정한 tree
- 신장트리(spanning tree): G 의 vertex를 모두 포함하면서 트리인 연결된 그래프
 - (c), (d)는 G 에 대한 신장트리
- 최소비용 신장트리(minimum spanning tree): 가중치의 합이 최소가 되는 신장트리
 - (d)는 G 에 대한 최소비용 신장트리

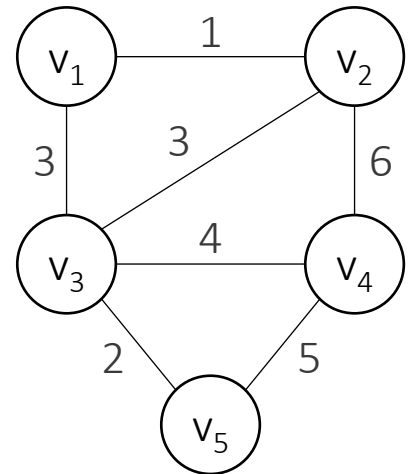
■ 최소비용 신장트리(minimum spanning tree) 문제

- 연결된 가중치 포함(weighted), 비방향(undirected) 그래프 G 에서 vertex는 모두 연결되어 있으면서도 edge의 가중치(weight)의 합이 최소가 되도록 edge를 제거하여 부분그래프를 만드는 문제
- 활용: 최소비용의 도로건설, 최소비용의 통신케이블 설치, 등
- 가중치의 합의 최소인 부분그래프는 반드시 트리가 된다
- 무작위 방법으로 찾으면, 최악의 경우 지수시간보다 나쁘다

■ 최소비용 신장트리 (Minimum Spanning Trees)

■ 그래프 $G = (V, E)$

- $V = \{v_1, v_2, v_3, v_4, v_5\}$
- $E = \{(v_1, v_2), (v_1, v_3), (v_2, v_3), (v_2, v_4), (v_3, v_4), (v_3, v_5), (v_4, v_5)\}$



■ 신장트리 $T = (V, F)$

- G 에 대한 최소비용 신장트리가 되도록 E 의 부분집합 F 찾기

$F = \emptyset$

// edge의 집합을 공집합으로 초기화

while (미해결) {

 부분적으로 최적인 edge 선택

// 선택과정

 if (그 edge를 F 에 추가하면 순환경로가 생기지 않음)

// 적절성검사

 그 edge를 F 에 추가

 if (T 가 신장트리)

// 해답점검

 문제 해결

}

■ 프림(Prim)의 알고리즘 (Minimum spanning tree algorithm)

■ 알고리즘

$F = \emptyset$

$Y = \{v_1\}$

while (미해결) {

Y 에서 가장 가까운 $V-Y$ 에 있는 vertex를 선택

 그 vertex를 Y 에 추가

 그 edge를 F 에 추가

 if ($Y=V$)

 문제 해결

}

// edge의 집합을 공집합으로 초기화

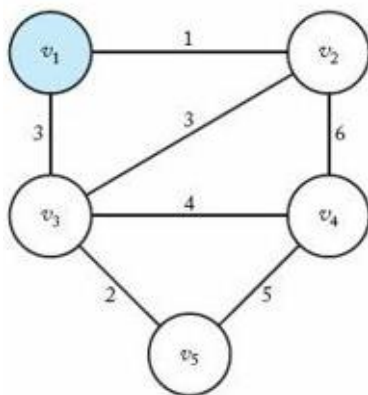
// vertex의 집합을 v_1 으로 초기화

// 선택과정 & 적절성검사

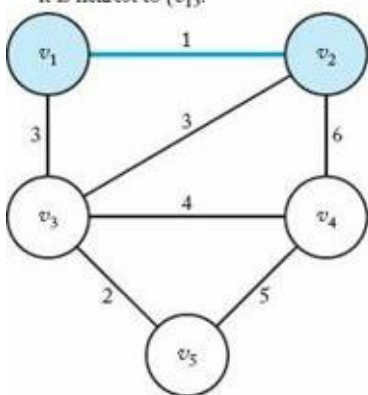
// 해답점검

- vertex를 추가해도 cycle이 생기지 않기 때문에 적절성 검사는 불필요함

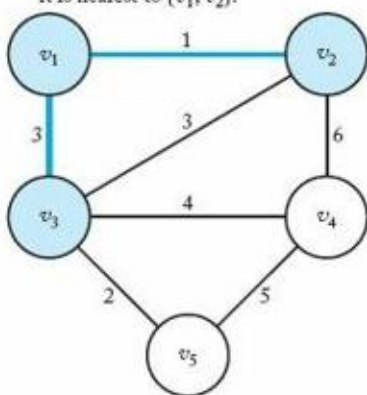
1. Vertex v_1 is selected first.



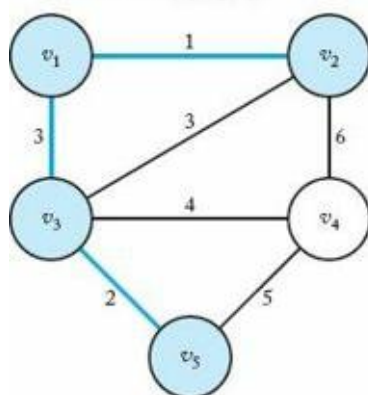
2. Vertex v_2 is selected because it is nearest to $\{v_1\}$.



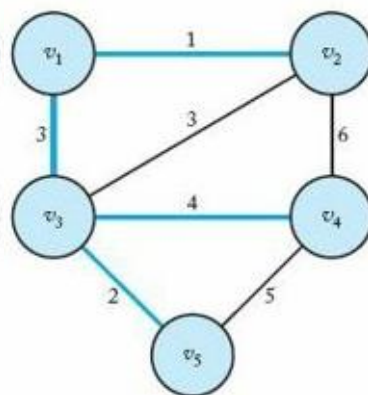
3. Vertex v_3 is selected because it is nearest to $\{v_1, v_2\}$.



4. Vertex v_3 is selected because it is nearest to $\{v_1, v_2, v_3\}$.



5. Vertex v_4 is selected.



■ 프림(Prim)의 알고리즘 (Minimum spanning tree algorithm)

■ Notation

- $W[i][j] = \begin{cases} \text{weight} (v_i \text{와 } v_j \text{를 연결하는 edge가 존재}) \\ \infty & (v_i \text{와 } v_j \text{를 연결하는 edge가 존재하지 않음}) \\ 0 & (i = j) \end{cases}$
- $\text{nearest}[i] = v_i$ 에서 가장 가까운 Y 에 속한 vertex의 인덱스
- $\text{distance}[i] = v_i$ 와 $\text{nearest}[i]$ 의 vertex를 연결하는 edge의 weight

■ Algorithm 4.1 설명

1) 초기화

- $Y = \{v_1\}$ 이므로, $\text{nearest}[i] = 1$
- $\text{distance}[i]$ 는 v_1 과 v_i 를 연결하는 edge의 weight

2) Y 와 가장 가까운 vertex를 찾는다

- min은 가장 가까운 distance
- v_{near} 는 그 때의 index

3) $\text{distance}[v_{\text{near}}] = -1$: 선택된 vertex를 제외시킨다

4) 확장된 vertex와 연결된 vertex의 정보($\text{distance}[i]$, $\text{nearest}[i]$) 업데이트

■ 프림(Prim)의 알고리즘 (Minimum spanning tree algorithm)

■ Algo 4.1 Prim's Algorithm

- 문제: 최소비용 신장트리를 구하시오
- 입력: n (vertex의 개수),
 W (connected, weighted undirected graph)
- 출력: F (최소비용 신장트리에 있는 edge의 집합)

```
void prim (int n, const number W[][],  
           set_of_edges& F) {  
    index i, vnear;  
    number min;  
    edge e;  
    index nearest [2..n];  
    number distance [2..n];  
    F =  $\emptyset$ ;  
    for (i= 2; i <= n; i++) {      // (1)  
        nearest[i] = 1;  
        distance[i] = W[1][i];  
    }
```

```
    repeat (n - 1 times) {  
        min =  $\infty$ ;  
        for (i= 2; i <= n; i++)    // (2)  
            if (0  $\leq$  distance[i] < min) {  
                min = distance[i];  
                vnear = i;  
            }  
        e = edge ( $v_{vnear}, v_{nearest[vnear]}$ )  
        add e to F;  
        distance [vnear] = -1;    // (3)  
        for (i= 2; i <= n; i++)    // (4)  
            if (W[i][vnear] < distance[i]) {  
                distance[i]=W[i][vnear];  
                nearest [i] = vnear;  
            }  
    }  
}
```

■ 프림(Prim)의 알고리즘 (Minimum spanning tree algorithm)

- Algo 4.1의 일정 시간 복잡도(Every-case time complexity) 분석
 - 단위 연산: repeat-루프 안에 있는 2개의 for-루프 안의 명령문
 - 입력 크기: 그래프의 vertex 개수 n

$$T(n) = 2(n-1)(n-1) \in \theta(n^2)$$